research · 2026-09-23

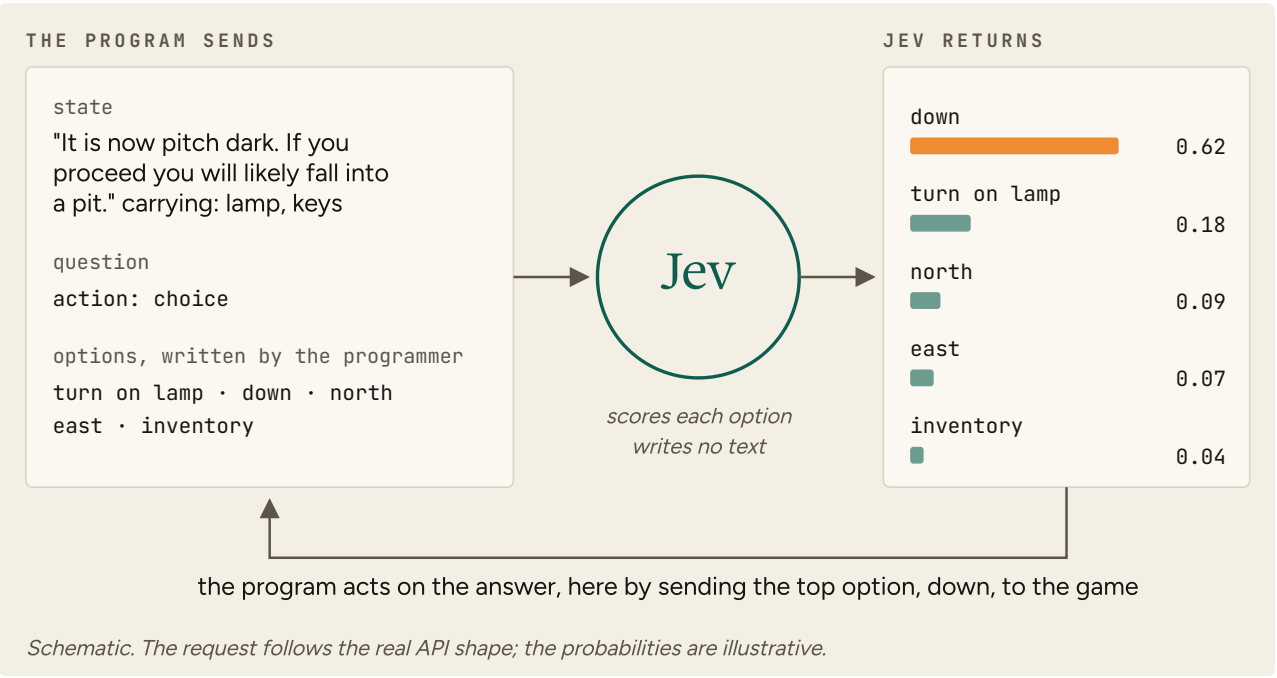
jev: three use cases, a ton of learnings

Adrian Tame Jacobo, writing for Plicara Labs · 2026-09-23 ·

Over about a week we pointed the Jev model at a deathmatch against a fruit fly brain, at 370 enterprise documents, at five text adventures spanning 1977 to 2007, and then tried to rebuild it on a laptop as well. We had a lot of fun doing this, and decided to share the learnings for anyone interested.

Jev launched on 15 September as a "System One model", a name TypeSafe takes from Daniel Kahneman's *Thinking, Fast and Slow*, where System 1 is the fast, automatic, intuitive mode of thinking and System 2 the slow, deliberate one. A program sends it a state and some typed questions, and it sends back probabilities over a fixed set of options. The code still decides what happens and the model just supplies numbers, at $0.042 per million tokens in, free out, in 70 to 500 milliseconds. It is named after William Stanley Jevons,[1] who noticed in 1865 that more efficient steam engines made Britain burn more coal rather than less, which is now called the Jevons paradox, and that says roughly what the company expects to happen once decisions get cheap enough.

ONE JEV CALL



Schematic. The request follows the real API shape; the probabilities are illustrative.

The calling program sends a state, a typed question and the options; Jev returns a probability for each option and writes no text; the program acts on the answer.

Something like this is very easy to wire into a program, which is the point! Diogo Almeida, TypeSafe's CEO, explains this quite succinctly in his recent Latent Space episode. It is also hard to evaluate, because there is no reasoning to read, only the option it picked and how sure it was. Then again, LLMs are not that easy to evaluate either, since their outputs are natural language, which is a series of progressively worse headaches to say the least.

The consensus in the community has been pretty loud and consistent. Laurie Voss at Arize asked whether decision models replace LLM judges and put the trade in numbers: on TypeSafe's own evals Jev gets 68% accuracy at $0.0004 and 0.4 seconds, GPT-5.6 Terra gets 68% for $0.03 and 10 seconds, and Opus 5 gets 73% for $0.18 and 38 seconds (Anthropic models, from our own testing, are terrible models to wire into systems; they are simply too expensive and not made for those use cases). So Jev is five points behind the best for about 440x less, a true paradigm shift that explains part of why people are so excited. This really unlocks things. He also took issue with the vendor's "can't hallucinate" line, which is fair and I actually agree with: type safety stops the model emitting anything outside the schema, and does nothing about whether the thing inside the schema is true. And JevBench adds a lot of information as well, which is that across its protocols Jev has landed anywhere from the low 60s to the mid 90s. So Jev's accuracy depends on how it is tested, and we picked four setups, mostly thinking about what this type of model unlocks. Two findings held in all of them.

The first is that the state and the option list are the controls, and the instructions barely are. Every prompt we wrote was neutral or worse. Taking an option away, trimming the state or changing what was on the menu moved the results in all three settings where we tried it, and the one puzzle the text adventures never cracked, lighting a lamp before walking into the dark, only fell once a second model started rewriting the menu.

The second is that the probability it hands back is not calibrated to correctness on our document-verification corpus. Whether it measures support in the supplied state is a hypothesis, not something this experiment establishes. **Among recovered in-text fields with Jev scores below 0.05, 93.8% were actually correct under the revised labeler, and in the games the most confident runs were the most stuck.**

At the job we cared about most, catching bad values in document extraction, a plain string comparison against a uniformly selected peer extractor catches an expected 78.3% of the errors against Jev's 42.7% on the revised comparison set, without additional model calls. The peer

calculation averages all available nonempty peers for each field; it is not a new two-extractor deployment test.

doom, against a fruit fly

The first one was not really an experiment, it was a fight, and the idea came straight from the horse's mouth: TypeSafe's launch demo already had Jev playing Doom at around ten decisions a second for about $7 an hour.

`doomfly` runs an actual fruit fly connectome as a Doom player. Frames stimulate modelled sensory neurons, activity runs through retained MaleCNS v1.0 wiring, and a fixed neuron-to-button interface turns, moves and fires. It is a map of a real fly brain, from the complete male *Drosophila* nervous system that Janelia, the MRC Laboratory of Molecular Biology, Cambridge and Google Research released on 3 September: over 166,000 neurons and about 125 million synapses, reconstructed one synapse at a time.

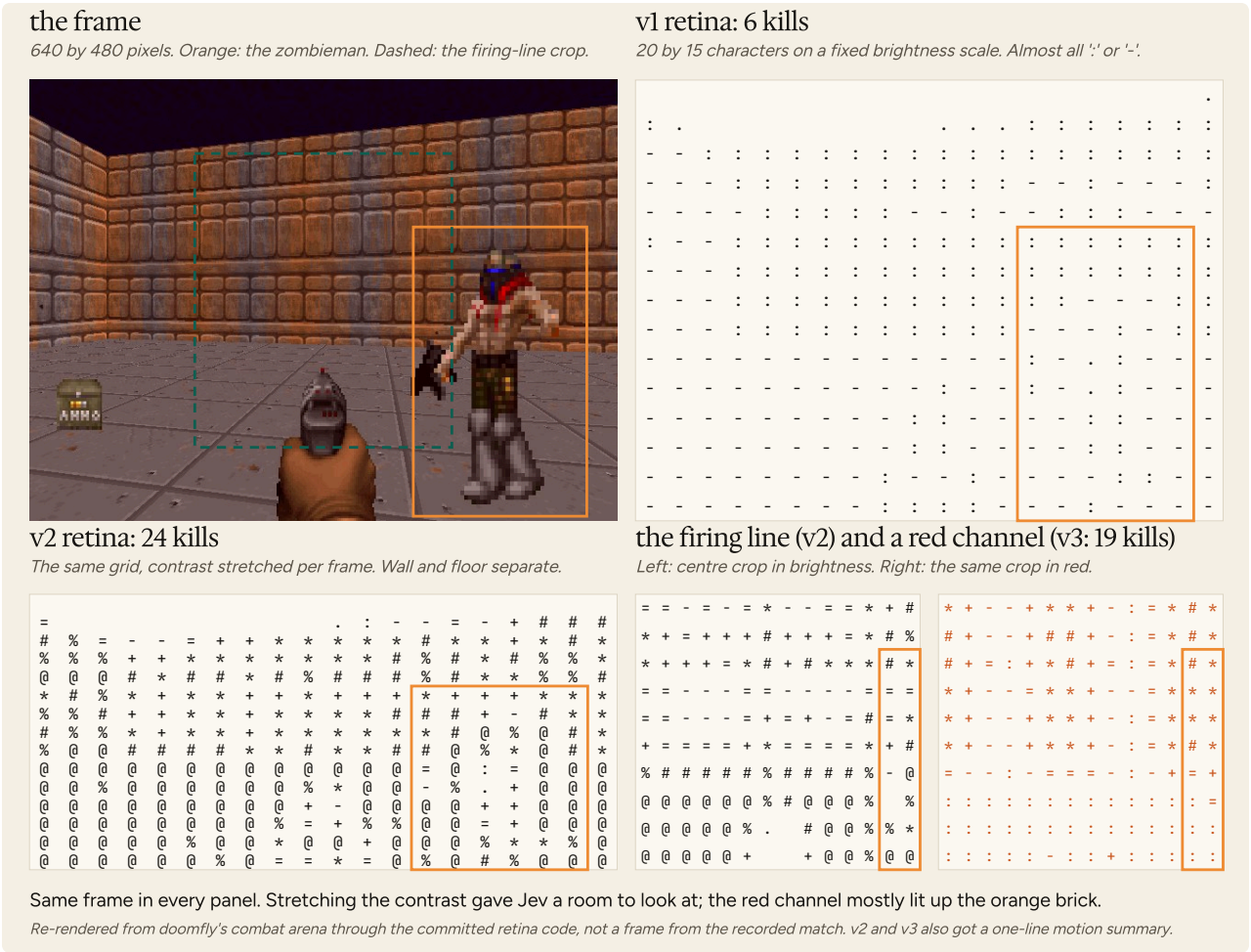
THE FLY'S VIEW



The first frame of a doomfly episode in the defend-the-center arena, with a cacodemon dead ahead. This is the fly's single viewport; the recording of the fly and Jev playing side by side did not survive.

Jev only reads text, so it got each frame as ASCII luminance grids plus a motion map, and answered turn, move and fire in one parallel call every five frames. Neither side saw engine state or enemy positions. Then the fly lived 60 to 70 seconds a life and Jev lived 10 to 22.[2]

WHAT JEV SEES



Same frame in every panel. Stretching the contrast gave Jev a room to look at; the red channel mostly lit up the orange brick.

Re-rendered from doomfly's combat arena through the committed retina code, not a frame from the recorded match. v2 and v3 also got a one-line motion summary.

One frame from the arena, then the same frame through each retina: v1's fixed brightness scale turns the whole room into ':' and '-', v2 stretches the contrast per frame and adds a crop of the firing line, and v3 adds a red channel that mostly lights up the brick.

The fly side though has not demonstrated learned survival, and its v6 candidate failed the visual, conditioning and survival validation gates. The fly surviving describes what happened in the runs

we recorded, and is not a claim about what a connectome simulation can do.

We also tuned the prompt and the like for Jev, and saw interesting results:

policy	what it saw	kills
v1	three thin questions	6
v2	five parallel judgments + a motion channel	24
v3	v2 + colour grid, memory of past deaths, damage reflexes	19

v1 to v2 quadrupled the kills, which felt like progress. v3 added a colour channel, a memory of where it had died before, and reflex rules for taking damage. **Every one of those sounds like an improvement and together they cost five kills.**

There is a name for this in the vendor's own limitations, which Langfuse relays: context rot, where accuracy drops as the state fills with material the question does not need. We had reproduced it by accident for about nine cents and put it down to feeding a text model a picture. We ran into it twice more later, in places that have nothing to do with Doom.

what we could recover from doom

The original head-to-head logs are still missing. A later experiment does have its event records: four seeds, each played for 90 in-game seconds with a prose retina and a same-day grid-retina control. Replaying those eight runs gives:

arm	total kills	total deaths	mean kills per run
grid retina, v3 control	83	45	20.75
prose retina, v5	119	43	29.75

That is 43.4% more kills across these four paired seeds, with v5 ahead on each seed. This is a small, exploratory comparison, not a general survival claim or a reconstruction of the fly match. Duration is recomputed from the recorded tics: the saved summary's `in_game_seconds` field measures only the final episode, so using it as the whole-run duration would be wrong.

370 documents and a headache

The idea is not ours: Cleanlab's Trustworthy Language Model already extracts fields, scores each one for trustworthiness, and sends the low scores to a human. We just swapped out the scorer. After any extraction, ask Jev one boolean per field, *is this value correct according to the document text?*, and treat that probability as the field's confidence. Every field in a parallel call; the model still has an input-token cost and a round-trip latency.

The following early controlled-set figures are historical exploratory reports, outside the recovered ledger replay.[4] On small controlled sets it looks fantastic. ExtractBench is LlamaIndex's schema-guided extraction benchmark, 370 enterprise documents over 4,869 pages, eight domains, 67 document types. Treat its ground truth as a perfect extractor, corrupt some fields on purpose, and the separation is almost silly: intact fields average 0.948, corrupted ones 0.014, and on the first run any threshold between 0.03 and 0.79 gives zero misclassifications. At 50 documents and 485 fields, no corrupted value was ever accepted at a sensible threshold, including eleven where the tampered value really did appear somewhere else in the document. Against a real extractor's real mistakes on seven documents it caught 86% of the failures and raised no false alarms at all.

When we dug we found that **most extraction errors start upstream in the text layer**. If OCR turns `MADELIN` into `ELINE`, any verifier reading that text will confirm the wrong name at p=0.98 and keep confirming it forever. So we asked Jev about the text layer itself, before extraction. On the benchmark's deliberately damaged PDFs against their clean twins, readability splits them 0.17 against 0.83, every corrupted file under 0.10 and every clean one over 0.74, and the pypdf-mangled document behind the `ELINE` case sits at 0.58, right where a gate catches it. Checking the text first and sending the bad files for better OCR would stop a whole class of confidently wrong extractions.

Then we ran it at scale, and the numbers got a lot worse. ExtractBench publishes a leaderboard of 44 systems. We took twenty of them, 370 documents each, labelled every scalar field against ground truth with the benchmark's own comparators, and asked Jev one boolean per field. That came to 96,680 verdicts on the first pass and 24,312 on a second, around 275,000 Jev calls over the campaign.

Pooled, at a 0.5 threshold:

metric	value
False alarms on good fields	26.5%
WRONG values detected	57.2%
MISSED values detected	15.1%

metric	value
INVENTED values detected	65.2%
All real failures flagged	41.6%

Which is a long way from 86% of failures at zero false alarms!

A free check explains most of the gap, though: we asked whether the predicted value's normalised string is anywhere in the document's text layer. For 62% of the good fields it simply is not: the value is on another page, or the text layer is mangled, or the document is a scan with no text in it at all. Nothing that reads text can confirm a value it cannot see, so a lot of that 26.5% is us measuring our own text extraction rather than the model! Split it by *presence* and it reads very differently. Wrong values that are not in the text get caught around 70% of the time. Wrong values that are in the text, meaning cross-slot grabs and plausible alternate readings and truncations, get caught around 32%, and that holds across all eighteen usable systems. The document decides how hard a value is to verify, whichever system extracted it.

We went after that 32% with better questions, borrowing from other people's work.

The nearest published thing to what we were doing is William Lyon's excellent knowledge-graph extraction post from 17 September. He has a local span extractor propose candidate relations and uses Jev to judge them, and his headline is an assertion gate, a supported-boolean plus a status choice across asserted, hypothetical, negated and forward-looking, taking edge precision from 0.279 to 0.404 without losing recall. His extractor was reading denials as facts. "Management has no plans to divest the Cascade brand" came out as an acquisition edge. Naming the modality in the options caught 25 of his 56 false edges.

We ported it. On our corpus it bought +3.6 points of detection for +3 points of false alarms, which is worth having and is nowhere near his jump. On why it is so different, it is that his errors and ours are different animals. His were unsupported or hedged claims, exactly what a semantic judge is for. Ours, once the not-in-text class is taken out, are mostly plausible readings of text that really does say something close to what was extracted. It is the same question but a much harder distribution, much smaller payoff.

Lyon also found that rewording one question moved his trap-versus-control separation from 0.07 to 0.63, and concluded that **ambiguity in a question comes back as a confident answer to the wrong reading**. We got a quieter version of the same pattern really. A completeness boolean, is this the *complete* value as stated, was our best single addition at +4 points, because it catches the truncations a plain correctness check waves through. Rephrasing "is X right?" as a choice

between X and "something else" did nothing whatsoever. Fusing three question shapes and flagging when any of them disagreed got us to 38% detection at 20% false alarms. They disagree almost independently, but that did not help, because two-of-three voting flags 87% of the good fields as well.

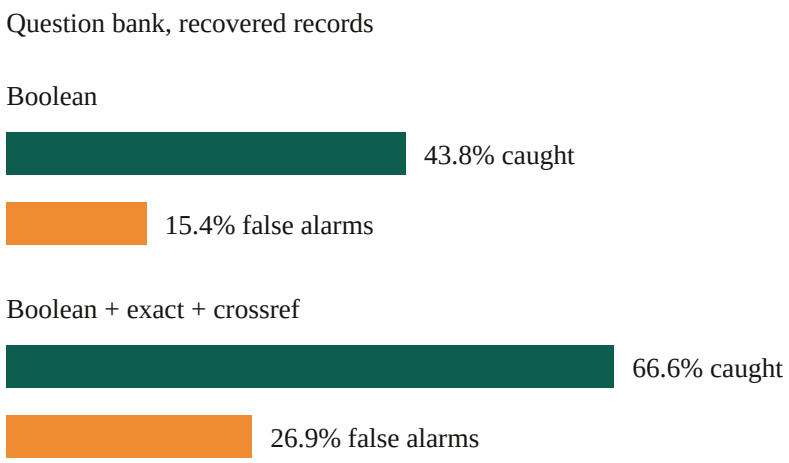
We were stuck there until we found BinEval, from Capital One, which evaluates by decomposing a criterion into atomic binary questions and measures how correlated those questions are with each other, because correlated questions add no marginal information. The measure is phi, which is just a correlation for yes-or-no answers: 1 means two questions always agree, 0 means knowing one answer says nothing about the other. Across their dimensions the mean phi is 0.38. We ran the same measurement on ours. Our boolean against the status choice was 0.79, against the three-way choice 0.86, against completeness 0.83. We had really built one question and three paraphrases of it, and therefore we were really not adding info when running multiple questions.

We then sorted the failures before writing any more questions. The misses were mostly values that had been cut short, values grabbed from a neighbouring field, formatting mismatches, OCR garbling and numbers that were slightly off, and the old question was good at some of those and hopeless at others: it spotted most values lifted from the wrong field and hardly any that had been cut short. Then we wrote one question aimed at each family:

- `exact` asks whether the complete value appears verbatim, for truncation and garbling.
- `crossref` asks whether the value in the text belongs to this field or a neighbouring one, for cross-slot.
- `numeric_ctx` asks whether the document states this number for this field, for numeric slips.

On the recovered E4 subset with both `exact` and `crossref` recorded, the revised labels give 43.8% detection for the original boolean and 66.6% for the fused rule. False alarms rise from 15.4% to 26.9%. These are paired scores on the same fields, including 587 WRONG and 25,584 OK fields. Missing responses and ambiguous revised labels are excluded; the old failure-family chart is replaced with this directly replayed comparison.

THE RECOVERED QUESTION BANK



Same scored fields; threshold 0.5; percentages of each label class.

Detection and false alarms on the same scored fields under revised labels; the fused rule flags when any of boolean, exact or crossref is below 0.5.

`crossref` also came with a real dial, one that trades catches for false alarms, which the rewordings never allowed. `exact` on its own is the precise one, flagging very few good values, which is what matters when the human review is the expensive part. Garbled values stayed the hardest, because they are OCR noise, and no question fixes a bad text layer.

Rewording a question does not make a new one, and phi shows that before any calls are spent. We then faced another hurdle though. We were sitting on eighteen systems' predictions of the same documents, so comparing them against each other cost nothing to check, so why not?

The first attempt came back at 100% detection and 0.2% false alarms, which is a huge screaming bug. When dealing with ML systems, too good to be true usually screams bugs. It counted a system's vote only when its value matched ground truth, so the answer was in the signal. We caught it, wrote it up as a circularity incident, and redid it as a plain string majority with no ground truth anywhere near it.

The original comparison table cannot be reproduced as written from the recovered snapshot. The original labeler gives 32.5% boolean detection across all recovered in-text verdicts, rather than the article's 37.4%. The later labeler changes 292 WRONG labels to OK and excludes 118 more as ambiguous. We therefore replace the table with a new, explicit comparison using that revised labeler.

All three rows below use the same 656 WRONG and 28,860 OK fields. One additional WRONG field has no eligible peer and is excluded from every row. A peer must return a nonempty value,

and the target extractor never votes on itself. The majority rule flags when the target value lacks a strict majority among peers, including ties. The pairwise row is the exact expected disagreement with a uniformly selected eligible peer, averaged equally over target fields.

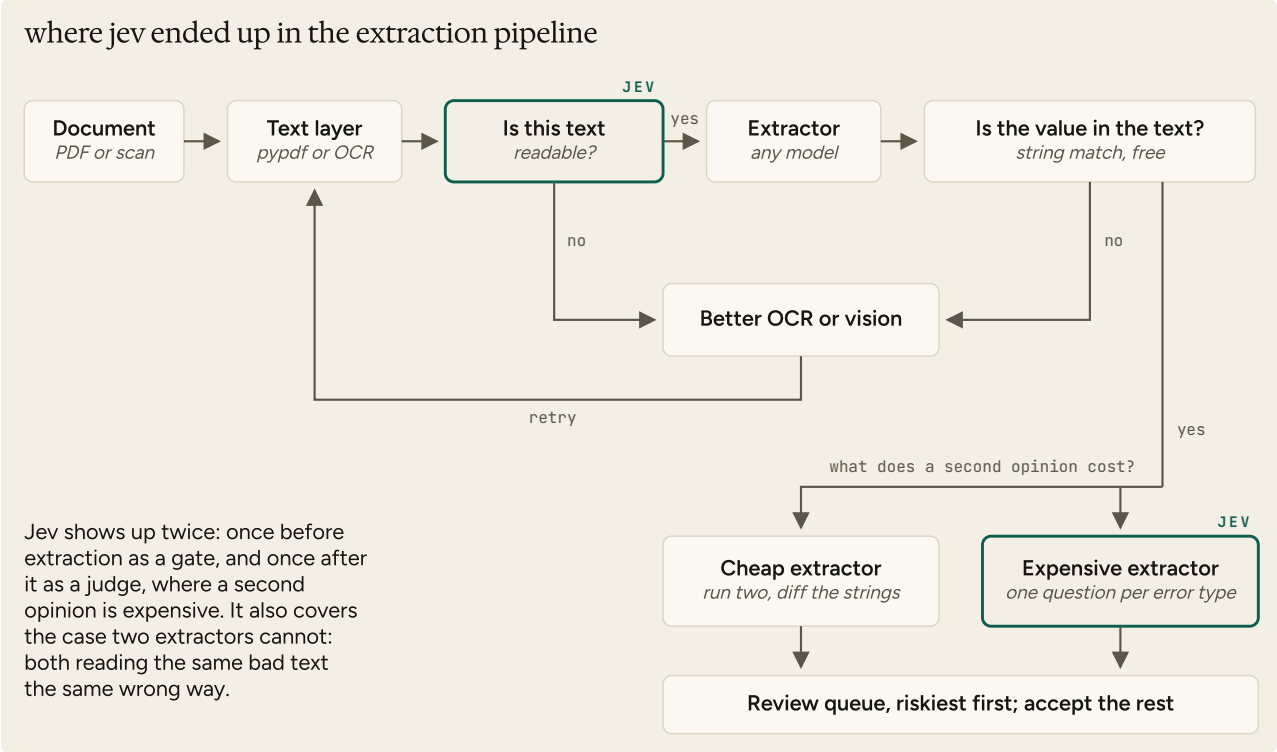
signal	catches bad fields	falsely flags good fields
Jev, single boolean call	42.7% (280/656)	15.2% (4,381/28,860)
strict majority of eligible peer extractors	84.9% (557/656)	1.1% (324/28,860)
uniformly selected peer, expected disagreement	78.3%	2.2%

The peer signals perform better on this revised set. They are computed from stored extractions, so this comparison adds no model calls; obtaining a second extraction in production still costs an extraction pass. Majority catches 317 bad fields Jev misses, while Jev catches 40 the majority misses. These are descriptive results on one corpus with correlated fields, not independent trials or a guaranteed production improvement. The revised labeler was developed after inspecting this benchmark, so these are retrospective scores rather than untouched holdout results.

There are related observations in other workflows, although they do not establish that this result generalizes. QuicqDev's eight-dataset comparison against classical ML **finds the same shape elsewhere, with zero-shot Jev beating eleven classical pipelines on IMDb sentiment at 96.3 against 88.4 and losing badly on tabular data.** JevBench has embeddings plus logistic regression ahead of zero-shot Jev on Banking77. **Lyon found that escalating his uncertain cases to Claude Haiku made precision worse, 0.404 down to 0.396, because low confidence meant the source text was genuinely ambiguous and a bigger model cannot fix ambiguity that lives in the document.** Everywhere we looked, when a cheap independent signal existed it won, and what the zero-shot decision model is actually good for is the case where there are no labels and no second system.

So the result moves Jev to a different place in the stack.

WHERE JEV ENDED UP IN THE EXTRACTION PIPELINE



A gate on the text before extraction, a free presence check after it, then either two extractors diffed or Jev's question set depending on what a second opinion costs.

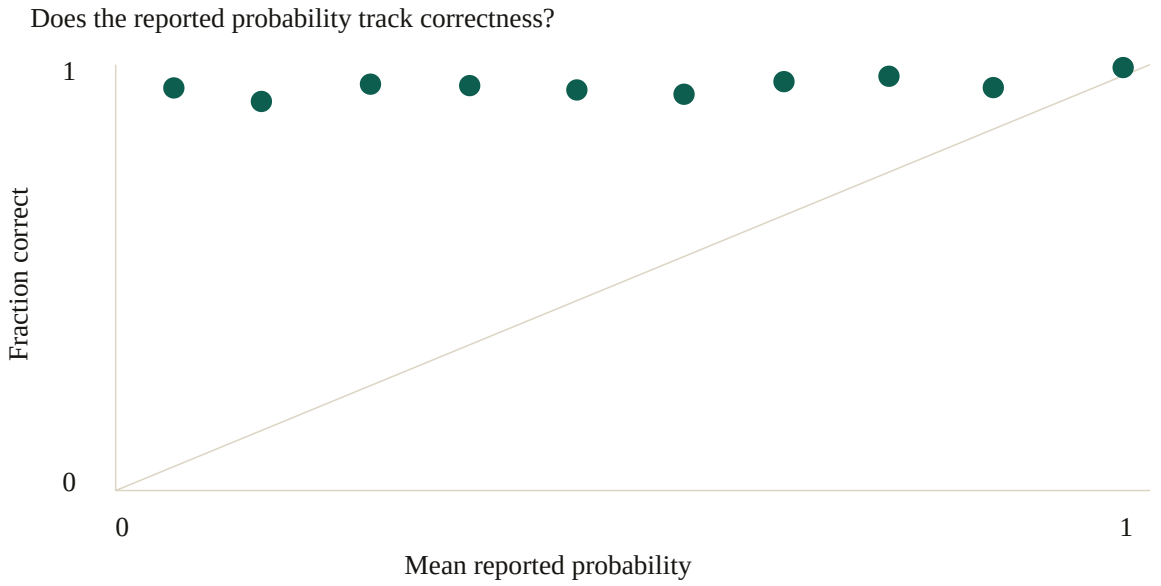
What decides the verification role is the price of a second opinion. A second extractor supplies a useful disagreement signal in this replay; Jev may still help when another extraction is expensive or when errors are correlated. The original projected F1 gains and per-document budget estimates have not been recomputed here and should not be treated as measured production improvements. A deployment needs its own costs, error mix and review policy.

Two other things came out of this as well, calibration and context rot, again!

The first is calibration. In the recovered in-text verdicts, 411 of the 438 fields scoring below 0.05 are correct under the revised labeler: 93.8%. There are no scores exactly equal to zero in this snapshot, so the original wording "at p=0.0" was misleading. A low reported probability does not behave like a calibrated probability of correctness here. The experiment alone does not establish why.

The figure now plots empirical correctness against mean reported probability in ten fixed bins, using all non-ambiguous revised OK/WRONG in-text fields. It replaces the old fitted-model transfer plot: the earlier holdout and external-extractor calibration numbers have not been replayed in this update and are withdrawn from the current quantitative comparison pending a matched analysis. No fitted calibration model is being evaluated in this figure.

PROBABILITY VERSUS OBSERVED CORRECTNESS



Recomputed from the recovered in-text verdicts with revised labels; each point is one fixed probability bin and the diagonal represents perfect calibration. This is a descriptive reliability plot on the observed corpus.

Someone then asked for the obvious product, one quality score per document from 0 to 100. We built it from the same signals and mostly learned which extractor had run. Within a single extractor it tracks the real error rate a little, moderately on the weak extractors and barely at all on the best one. It can sort a queue by risk, but it cannot say a document is 92% accurate. The per-field flags that come with it turned out to be the more useful thing anyway.

The second is context rot again. We packaged the same evidence for the same question three ways, and the enriched state, more material and more windows and more context lost 7.6 percentage points of detection in the paired replay: 43.1% to 35.4% under revised labels. False alarms also fell, from 15.2% to 14.2%, so this is a trade-off rather than evidence of uniformly worse decisions. It's interesting to compare that to JevBench's most striking finding, which points the other way: on 5,733 phishing emails the same model went from 85.7% to 98.4% recall on an evidence-enriched prompt. Enrichment is the biggest lever they found and it cost us some. There is probably something to be said here about prompt tuning, but honestly, that game of cat and mouse is not something we went too deep into.

Then again, their enrichment added criteria informed by labelled errors, which is supervision folded into a prompt. Ours added context the question did not need. What mattered was how relevant the added material was, not how much of it there was, and the two look the same until someone measures them, or until a strategy from another problem carries over.

five text adventures, and a lamp

At this point we had two findings that kept recurring and no clean way to separate them and both settings confound whether the right answer was available to the model at all with whether it picked it.

A text adventure was a good next fit, even if we didn't really imagine it when we started working on this. Jev cannot generate text, and in an adventure game the right move is very often a novel two-word command, so it has to be handed a candidate list every turn: universal moves, plus verbs triggered by whatever nouns the game just printed. That is pretty crippling. It is also what makes the setting diagnostic, because the option list is a plain list, and every result splits cleanly into whether the right move was on the list and whether it chose it.

This is a well-worn research area and our limitation is the standard move in it. Jericho, from Hausknecht et al., *Interactive Fiction Games: A Colossal Adventure*, AAAI 2020, supports 32 IF games and exists precisely because combinatorial action spaces defeat naive agents, and it cuts them down with template extraction. Microsoft's TextWorld (Côté et al., 2018) does the same for generated games. Our hand-built candidate list is a cruder version of Jericho's templates. We are not competing with any of that: there is no Jericho score here, no RL, no trained agent to compare against. It is a test of one kind of model in a setting where every possible action can be listed.

The first live run was very charming. From room text alone, with no hints, it produced the exact opening checklist a human walkthrough of Colossal Cave Adventure gives: `take keys`, `take lamp`, `turn on lamp`, `take bottle`, `take food`, `eat food`. Then it spent turns 15 through 60 going `in`, `look`, `out`, `look` between the well house and the road outside. Aced the tutorial and then completely lost the plot.

So we went and played the prompting game. We tried providing instructions and hints, stuff like that, and spent the better part of a day testing out these strategies to see if we could eke out a little performance.

Every single instruction-level change was neutral or harmful. "Press deeper" was the worst intervention in the whole campaign: 271 picks of `down` (pretty funny in context), 110 of `in`, and it never left the starting room. Telling it not to backtrack did not make it go forward, it made it hammer the direction the instruction praised into a wall. The anti-loop instruction got 58 picks of the magic words `xyzzy` and `plugh`, which every other variant ignored completely, and no progress.

What really worked was deleting one option, which is quite interesting.

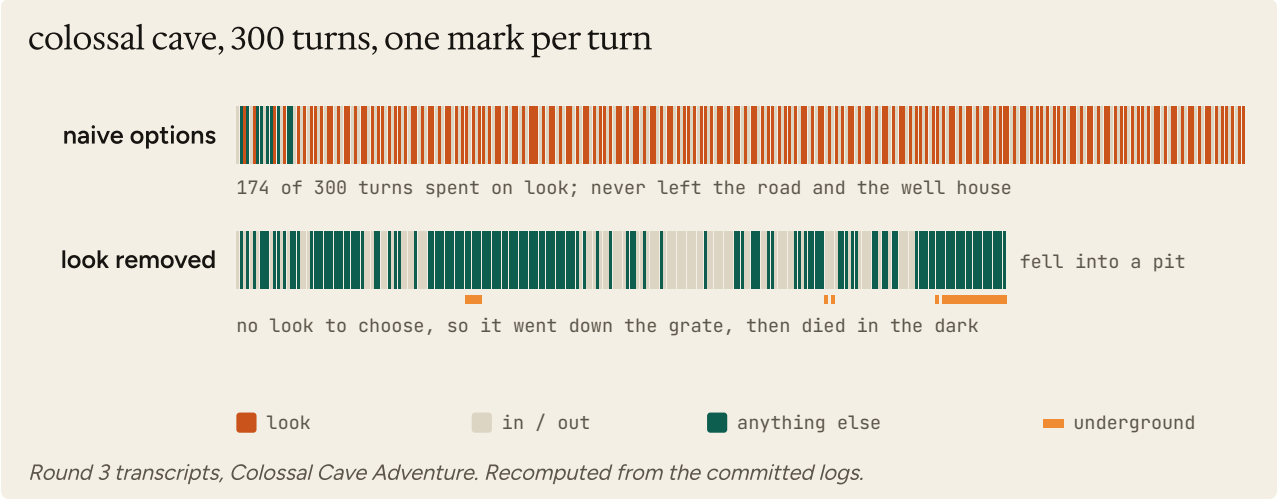
`look` costs a turn and returns text the model already has in its state. Take it out, widen the state window from three rooms of history to ten, and that config won 7 out of 7 paired comparisons against the naive baseline, in every game and both rounds it ran in:

game	round	baseline	config	delta
dreamhold	r3	20	36	+16
dreamhold	r4	23	26	+3
905	r3	14	25	+11
905	r4	13	25	+12
lostpig	r3	11	15	+4
lostpig	r4	13	15	+2
adventure	r3	3	7	+4

One honest footnote on that table. Only Adventure's numbers are rooms. The other four games print no room title under our interpreter, so their counts are distinct responses, which is to say variety, and by the games' own scores the config made no difference on Lost Pig or Dreamhold. The runs stopped going around in circles everywhere, and Adventure is the only game where that turned into getting further.

The baseline spent 174 of its 300 turns on `look`, which is 58% of the game re-reading text it already had in front of it. Dreamhold's round-four baseline did the same thing at 79 turns.

COLOSSAL CAVE, ONE MARK PER TURN



Round 3 transcripts, Colossal Cave Adventure. Recomputed from the committed logs.

The naive run is a wall of look and in-out shuffling; with look removed it goes down the grate.

It is specifically `look` and not clutter in general, which surprised us. A variant that padded every list with six plausible no-ops, `sing` and `meditate` and friends, landed on the baseline's numbers exactly: 3 rooms, 0.19 repeat rate, 0.55 confidence. Jev ignored the obvious filler. The option that hurt was `look`, which sounds productive and wastes a turn. What this is a lesson of in the broader Jev ecosystem is more complex and hard to answer, but there is something that seems quite important here.

Lyon arrived at the same place from the opposite end: the judge is only as good as its queue. A perfect judge pointed at the wrong candidate set bought him no F1 at all. We saw the same thing: one plausible no-op on the list took more than half the baseline's turns.

Confidence behaved the same way it had on the documents. Across 29 runs, mean confidence correlates -0.21 with how many distinct places or responses a run reached and +0.34 with the immediate-repeat rate. The most confident runs were the most stuck, and the ones that got somewhere sat between 0.35 and 0.50. The same way p=0.0 on a document meant no visible support rather than wrong, **high confidence in a game meant it kept seeing the same narrow state, not that it was doing well**. Bloss0m's agent-runtime write-up puts the operational version of that as a flat rule, which is that confidence is not permission.

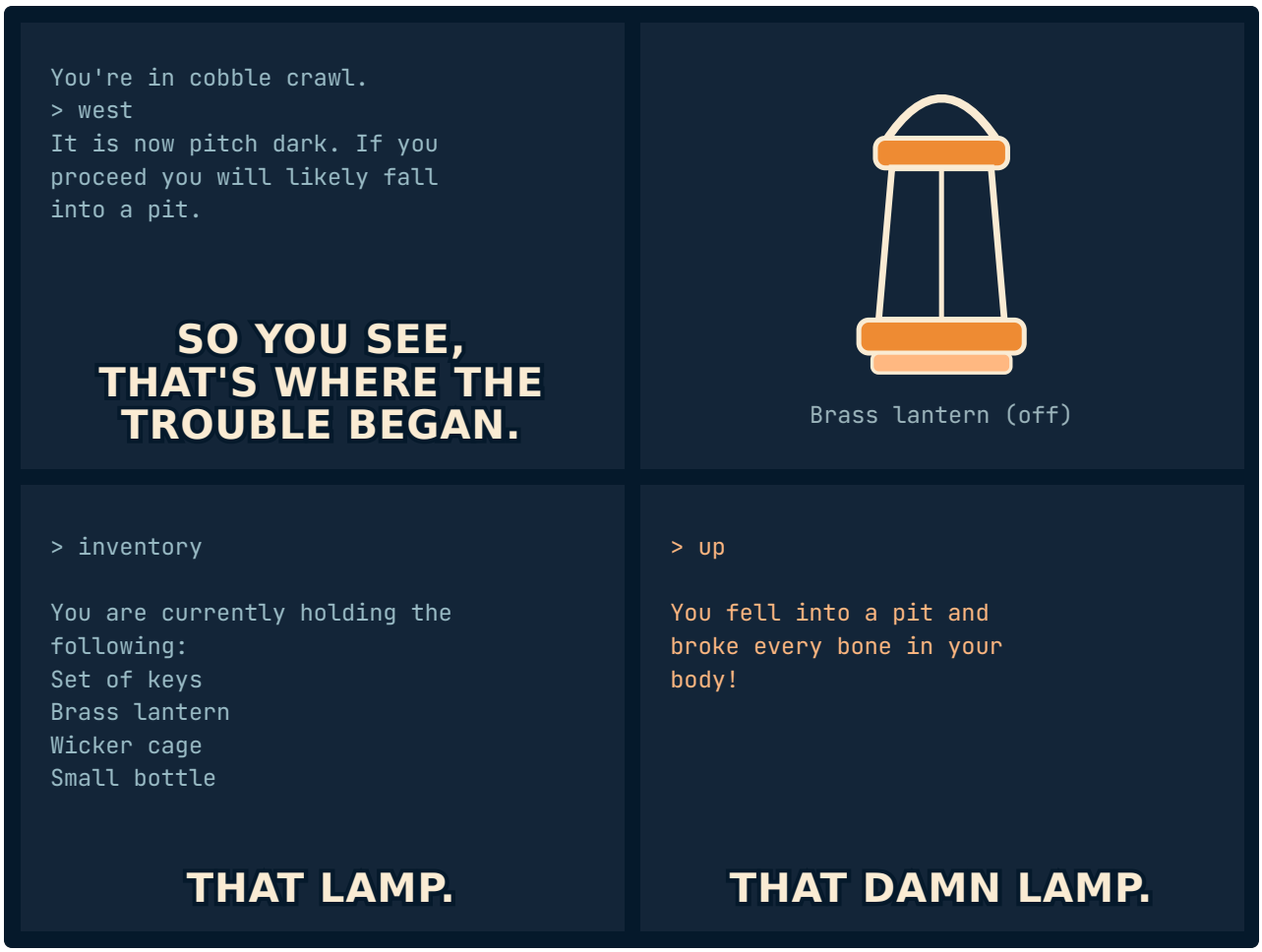
We also caught a pretty interesting bug along the way, round 3 could be interpreted as "Jev cannot play Curses": three rooms, a 0.98 repeat rate, 281 picks of `north`, and the highest mean confidence in the campaign. All of that was the model talking to a game that had ended! On turn 1 it picked `down` through an open trapdoor, which ends Curses instantly at 0 out of 550 with the rank of *hapless Tourist*, and our endgame detection only matched one dialect of death message. The harness then spent 299 turns arguing with a post-game menu (pretty funny it got stuck there). Round 1 had the same bug with a better punchline: the goal prompt worked, drove straight

at the cave entrance, and the game asked a yes/no clarifying question that was not on the candidate list. It burned 294 of 300 turns on "Please answer the question." Again, this probably has lessons for bigger problem spaces, and agentic engineering. What happens when an agent is not given a tool it needs, or its sandbox restricts it in a way that it cannot perform the necessary actions to complete a task? Lobotomized models are a real unexplored area of AI systems, and things like the OpenAI and Hugging Face incident in July, where agents in a cybersecurity evaluation broke out of their sandbox and into Hugging Face's systems looking for the answers to their test, show the lengths models will go to to try and find solutions to the problems they are presented.

That failure has a name in the ecosystem too. Langfuse's guidance, out of Good Start Labs' production use, is that **Jev cannot abstain**: a forced binary with no unknown option makes it pick the least wrong answer, so the design needs an escape hatch. Our yes/no deadlock is the same problem: the game wanted an answer that was not on the list. An explicit "absent" option on null questions gave zero false flags across 22,435 correct absences. The same escape hatch on value questions cost 6.6 points of detection, because the model picked it even when the value was there but wrong.

Which brings us to the lamp.

THAT DAMN LAMP, IN THE FOUR-PANEL "THAT DAMN SMILE" FORMAT.



Four Adventure runs got into the cave. All four had picked up the brass lamp. None of them ever turned it on. Each one was told, in the state text it had just been handed:

```
It is now pitch dark. If you proceed you will likely fall into a pit.
```

Each one stepped anyway, at p=0.87, at p=0.76, at p=0.31, and got:

```
You fell into a pit and broke every bone in your body!
```

`turn on lamp` was on the candidate list every one of those turns. The one variant that seriously tried to light the lamp, nine picks of `turn on lamp`, never left the first room. And the very first sixty-turn run, the one that stalled at the front door, did light it, on turn 7. The runs that got underground never lit the lamp, and the run that lit the lamp never got underground. I think the interesting lesson here is that this is not a thinking model. We have become so used to thinking

models, that can in a sense reason through a task or are given ample space to put their thoughts in order, but this model is quite different. It is in a sense stateless, all the cache (question) is passed on as a single fragment, and while the transformer architecture is very powerful, all it really can do in this scenario is predict the next token. This is much closer to a stochastic parrot (Bender et al., 2021) than one of the uber powerful new models out there like Astra or Opus 5.5. They behave differently and they need to be treated and worked with very differently. Things that seem obvious (massive hint: use the light source) do not actually add anything interesting *because the model cannot generate an intermediate state where that information becomes useful*. Every individual decision there is defensible and in sequence kills the run. Nothing in the loop carries "the thing I am holding solves the thing about to kill me" from one turn to the next. No prompt fixed it. What was missing was a second question: does anything in the inventory deal with this?

So then, maybe we are asking too much of little Jev, the system 1 thinker. Maybe it just needs a big brother to hold its hand as it goes through the dungeon.

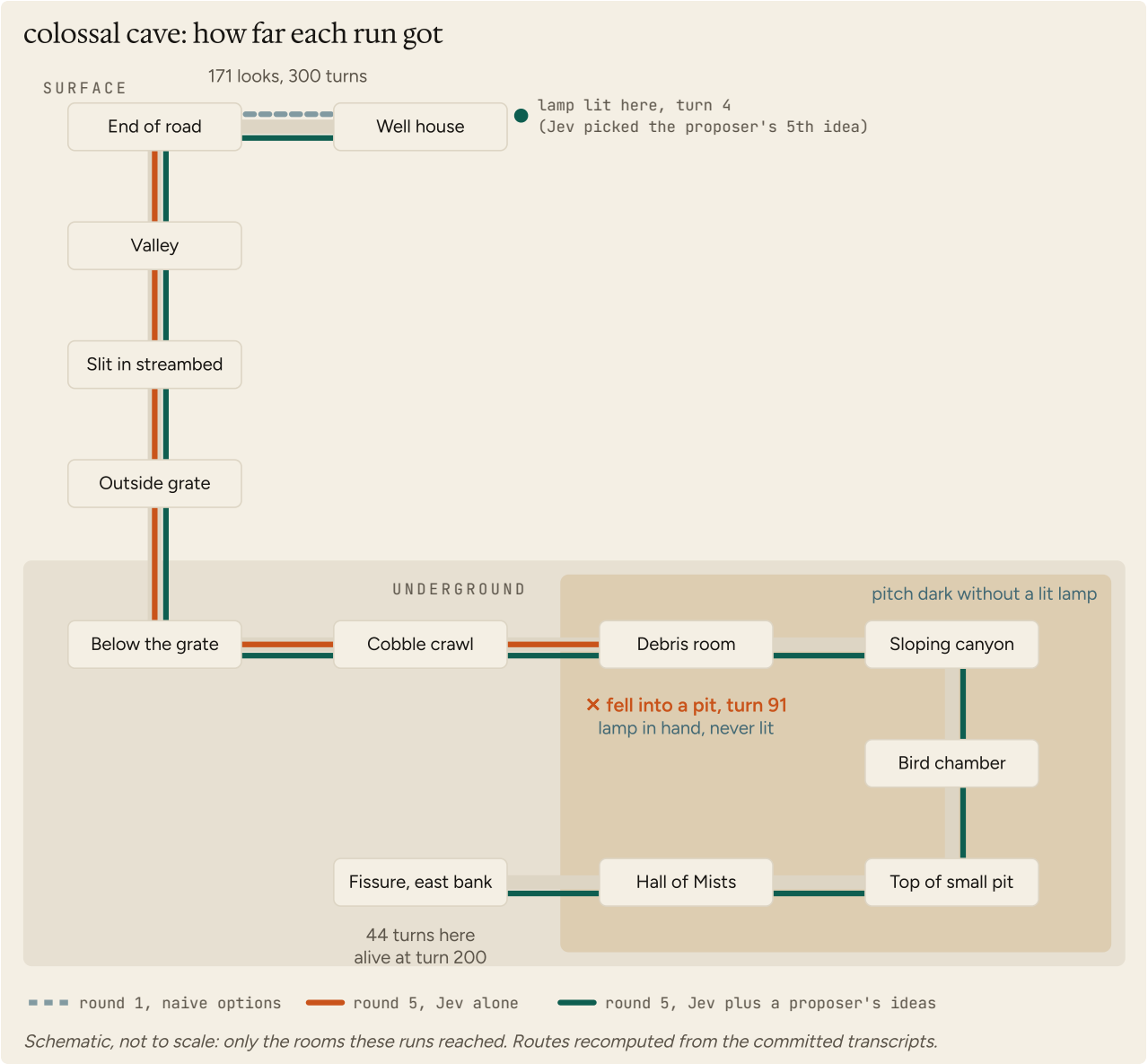
Round 5 put a generative model in front of Jev. We intentionally chose a not SOTA small open model, `gpt-oss-20b` because cheaper/smaller models show breaks more often. If it works with a small, older model, it's likely the newer models will be able to handle the task as well. It reads the room text and proposes eight commands a turn, and then one of three things happens: Jev picks among the proposals, or Jev picks among the proposals merged with the old option list, or the proposer's own first idea gets played with no Jev at all. A Jev-alone run on the same day is the reference.

Adventure's score had not moved off 32 of 430 in any run of any round. The merged arm scored 59,[3] and this is its turn 4:

```
[004] 'light lamp' p=0.39 (22 options) proposed=["open door", "go
outside",
      "drink water", "eat food", "light lamp", "use lamp", ...]
```

It saw the light! After that it saw no pitch-dark warnings for the rest of the game. It went through the Hall of Mists, spent 44 turns at the fissure, reached the bird chamber, and was alive at turn 200 after 64 turns underground, **deeper than anything in five rounds**. Jev alone, in the same round, died on turn 91 in the same pit as every other time.

COLOSSAL CAVE, HOW FAR EACH RUN GOT



Round 1 never leaves the road; round 5 Jev alone dies in the dark past the cobble crawl; round 5 with the proposer lights the lamp on turn 4 and reaches the fissure.

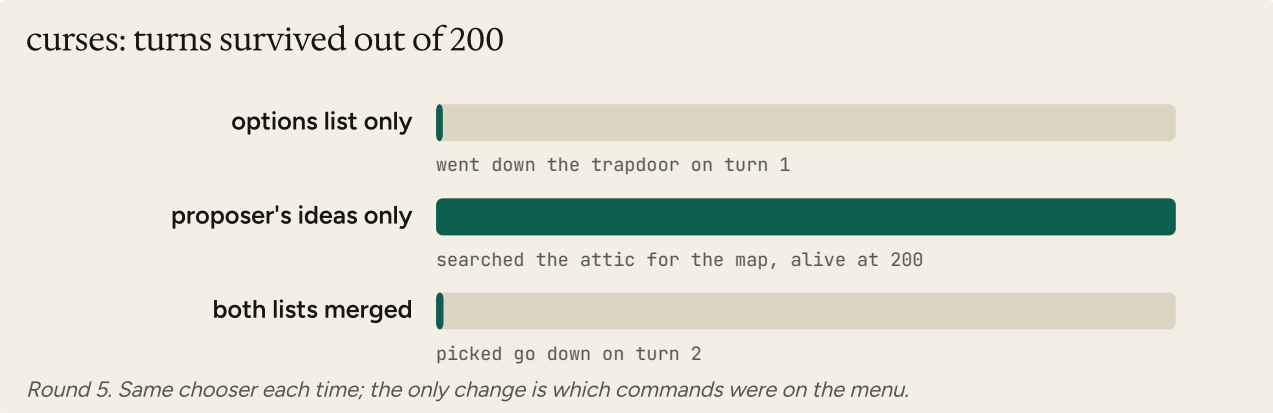
It needed a big brother. `light lamp` was the proposer's fifth suggestion, so the arm that plays the proposer's first idea would never have taken it. The arm that saw only the proposer's suggestions lit the lamp three times and never found the way down, because the route underground, `downstream` and `unlock grate`, came from the old hand-built list. The proposer came up with the lamp, the old list had the way down, and only the arm that showed Jev both lists got both.

Do not get excited, the rest of the round argues against reading that as a general result. Lost Pig went from 1 of 7 to 2 of 7 in all three proposer arms, including the one that never consults Jev, so that gain belongs to the proposer. Outside the lamp moment, Jev picking from the proposals

lands on the same scores as the proposer playing its own first idea, even though it agreed with that first idea on only 14 to 33% of turns. The proposer's commands also often failed: between 10% and 57% of them were rejected by the game's parser, against 0 to 8% for the hand-built list, and Jev cannot tell which suggestions will parse.

Curses shows what the option list does most clearly. Alone, Jev takes `down` through the trapdoor on turn 1 and loses, for the fifth run running. Given only the proposer's suggestions, it spends 200 turns in the attic hunting for the map, which is what the game actually wants. Merge the lists back and it takes `go down` on turn 2. Nobody scored a point in any of those runs, but survival is now a clean function of whether the trapdoor is on the menu.

CURSES, TURNS SURVIVED OUT OF 200 WITH THE OPTIONS LIST, THE PROPOSER'S IDEAS, AND BOTH MERGED.



Round 5. Same chooser each time; the only change is which commands were on the menu.

So our guess about a missing question was wrong. `turn on lamp` had been on the list all along, one of dozens of options, and what changed was a generative model suggesting it at the moment it mattered. Jev cannot come up with that on its own.

building jev-stein

`jevstein` is a tier-0 reimplementation, a local open System One model speaking the same `/v1/systemone` shape with a frozen 4-bit 4B instruct backbone on Apple Silicon. Each option is scored as a continuation under a restricted softmax, so nothing is sampled, nothing is decoded, and no text is generated. It has the same interface as Jev and none of its training; calibration and outcome training would come next.

Other people are building the same thing, and the most useful reference was Bespoke Labs' Nimble, an open recipe for this exact thing: typed local decisions read out as one answer-code

token, trained with LoRA over the allowed candidate logits. It is far from alone, since Von, open-jev, Laya and a handful of others all speak the same `/v1/systemone` shape, each with a different idea of what should sit behind it. Nimble's numbers are the honest benchmark for anyone trying an open System One model. On a 324-example holdout, base Qwen3.5-9B agrees with the reference labels 66.4% of the time, their fine-tune gets 90.1%, and Jev 1.13.0 gets 93.2%. So an open recipe gets close to Jev, but not all the way. The benchmarks for this space are entirely new though, so do take these results with a sizable grain or bottle of salt.

Their notes also fixed something we had already tripped over. We split state from question suffix at a text boundary, and tokenizer merges ate it: a JSON state ending in `"}` merges with the newline after it, and every question quietly fell through to the slow path. We fixed that by splitting at a special token that never merges. Nimble's version is cleaner, which is to compute the longest common token prefix across all the field prompts and reuse exactly that.

There is a pretty vast graveyard of ideas though. A softmax where a log-softmax belonged, which produced flat distributions that looked like three different prompt bugs. And batching the question suffixes into one pass, which came out 7x slower end to end because the state prefill dominates everything else. That second one is interesting because Nimble's parallel scorer does the batched design and it works for them, since their suffixes are short and the readout is a single token, so the fix is to adopt answer codes and keep batching.

Getting a Doom-scale decision fast enough took it from 29.1 seconds to 3.35, an 8.7x improvement, and what is left is state prefill rather than branching.

what the week taught us

One important lesson is that the control surface is the state and the option list. Doom v3 added a colour channel and a death memory and lost five kills. The enriched document state added context and lost 7.6 percentage points of detection while reducing false alarms. The adventure baseline carried one plausible-sounding option and lost 58% of its turns to it, while every prompt we wrote was neutral or worse. That is context rot, which the vendor documents and Langfuse relays. JevBench's phishing result shows that the right extra material helps a lot, so what matters is relevance, not volume. Editing the instruction text is usually the wrong place to tune one of these.

The probability needs task-specific validation before it can be treated as confidence. Arize's spam numbers show excellent calibration, the recovered in-text verdicts show 93.8% of fields correct below p=0.05 under revised labels, and in games high confidence marked the runs that were most stuck. Evidence quality may help explain that difference, but our replay does not

isolate the cause. A probability that works on one task cannot be assumed to be calibrated on another. Either way, Bloss0m's rule applies: confidence is not permission.

Also, it is worth it to test against the dumb baseline before building anything more complex. A pairwise string comparison between two extractors beat the model at its flagship use case, 78.3% expected detection against 42.7%, with 2.2% versus 15.2% false alarms on the revised common set, without additional model calls. QuicqDev found classical pipelines winning on tabular data, JevBench found embeddings and logistic regression ahead on Banking77, Lyon found escalation to a bigger model actively harmful. The model is still useful, just somewhere else: gating before expensive work, triage, the shared-error class no cross-check can see, and the case where labels and second opinions genuinely do not exist.

Also, most of what looks like a model failure is a harness failure (likely a lot to learn here about bigger systems). JevBench's editorial line hints this, that a score without its protocol is not a claim and a public repository is not a reproduction, and the cheap internal defence is making the analysis script report where things actually ended and how many turns came afterwards, so no run can quietly pad its own numbers.

We also found that the loop around the model is where the intelligence has to live. Providing the right context to these types of models will be really relevant, because of the fact that they are stateless. Not only that, but we have not even done adversarial designs in here, if I were to introduce a "TERRIBLY NICE STAFF" to a system that is classifying on positive or negative feedback, the model might come back with a nice or naughty, but is it worth anything?

And the smaller lessons, which are scattered through everything above and are just as useful:

1. Ambiguity in a question comes back as a confident answer to the wrong reading. A good question has only one way to be read.
2. Rewording a question is not designing one. How correlated the questions are can be checked before paying for all of them.
3. Sorting the failures before writing questions lets each question aim at one kind of failure.
4. Too good to be true is a bug until proven otherwise. Our 100% baseline had the answer key inside it.
5. Different models have different blind spots. When extraction is cheap, two extractors and a diff come before any judge.
6. A verifier adds the most where the base system is worst. On frontier extractors there was barely anything left to catch.

7. Ranking travels and calibration does not. The probabilities need refitting on a labelled sample of whatever is actually being watched.
8. A request for one quality number per document mostly returns which system produced it.
9. A way to say "none of these" belongs only on questions where none of these is a real answer.
10. Obvious filler is harmless. An option that sounds useful and does nothing will eat half the run.
11. The judge is only as good as its queue. A perfect chooser cannot pick a move that is not on the list.
12. An agent missing the tool it needs loops, or goes looking for a way around the harness.
13. This is not a thinking model. A hint that needs an intermediate thought does nothing; the action has to be on the menu instead.
14. It works best paired with something that can imagine. In Colossal Cave a small, cheap proposer plus the chooser beat either one alone, and small models are where the breaks show up first.
15. Generated options often fail to parse, and the chooser cannot tell which ones will.
16. The fix is usually in the state, the options or the harness, not the prompt. Every instruction we wrote was neutral or worse.

In short, **a model that only chooses is only as good as the choices put in front of it**. It also makes debugging easy, because what went wrong is sitting in the option list.

---

1. We love the recent naming conventions for ML models, like Anthropic's Claude, and Jev for Jevons. It's nerdy in the best way. ↩

2. These historical numbers come from our write-up at the time. The original head-to-head run data was not found when the research machine became available on 27 September 2026, so the survival and kill counts in this section remain unverified. Later Doom experiments are documented separately and cannot stand in for this head-to-head run. ↩

3. The round 5 result is from a single run of each arm. ↩

4. The early corruption tests, original question-shape summaries, fitted-model calibration/transfer analysis and projected deployment improvements are not covered by this replay. The recovered JSONL files contain per-field probabilities and recorded outcomes, not complete HTTP requests and responses; replaying them does not reproduce provider inference. The correction ledger lists the claims reproduced, replaced and still unresolved. ↩